Predicting YouTube Video Views Using Machine Learning: An End-to-End Regression & Production MLOps Approach

Part II: Production MLOps Engineering Lifecycle (Labs 6 to 13)

Author: Koppuravuri Sri Hinduja (241FA18284)

Email: koppuravurisrihinduja@gmail.com

Institution: Department of Computer Science & Engineering

Date: November 2026

Repository: https://github.com/Srihinduja27/youtube-view-prediction

---

Abstract

Following the empirical validation and hyperparameter tuning of baseline ensemble regression models (Labs 1–5), wherein the Tuned Random Forest Regressor demonstrated strong predictive performance (R^2 = 0.7414 on transformed log-scale views and R^2 = 0.9382 on original view counts), this work presents the operational transition of the model artifact into an automated, enterprise-ready Machine Learning Operations (MLOps) architecture (Labs 6–13). The operationalized lifecycle implements: (1) centralized experiment tracking and model lineage governance via an MLflow Model Registry (`YouTube_Views_Production_Model`) with automated Champion vs. Challenger stage promotions; (2) a modular, six-stage production data and training pipeline featuring cryptographic SHA-256 batch fingerprinting and Pandera data contract validation; (3) continuous integration quality assertion gates enforcing strict generalization ($R^2 \geq 0.65$); (4) defensive fault-tolerance and failure injection testing verifying 100% fail-fast error interception across corrupted ingestion vectors; (5) a high-throughput, low-latency RESTful API serving microservice developed with FastAPI, featuring asynchronous JSONL inference logging and sub-100ms response latencies; (6) containerization with hardened non-root Docker images and Kubernetes orchestration manifests featuring 2-replica load balancing and liveness/readiness health probes; (7) a multi-modal statistical drift monitoring subsystem employing two-sample Kolmogorov-Smirnov (KS) hypothesis testing across input features, prediction distributions, and ground-truth concept drift; and (8) a closed-loop automated monitoring trigger orchestrator that detects distribution degradation and conditionally executes pipeline retraining upon arrival of new cryptographically validated data batches. The complete system establishes an MLOps Level 2 operational maturity for predictive regression modeling on YouTube trending metadata.

Keywords: MLOps, Model Registry, MLflow, FastAPI, Kubernetes, Docker, Data Drift, Concept Drift, Kolmogorov-Smirnov Test, Quality Gates, Automated Retraining, Fault Tolerance.

XXV. Transition to Production MLOps and System Architecture

While traditional machine learning workflows terminate at offline model evaluation and static test predictions (Sections I through XXIV), deploying machine learning models into production demands rigorous software engineering and automated lifecycle operations (MLOps). Machine learning systems in production are subject to silent failures, including schema changes, missing data, upstream API breakages, covariate shift, and concept degradation over time.

To bridge the gap between offline experimentation and dependable production service, the system was re-architected into a modular, decoupled, and continuous MLOps ecosystem. The transition introduced eight foundational operational pillars:

1. **Automated Lineage & Cryptographic Ingestion:** Tracking raw data batches with SHA-256 hashing to ensure data reproducibility and prevent duplicate processing.

2. **Data Contract Schema Validation:** Enforcing strict schema constraints via Pandera prior to downstream preprocessing.

3. **Reproducible Pipeline Decoupling:** Isolating preprocessing, training, and evaluation into independent stages with serialized artifacts.

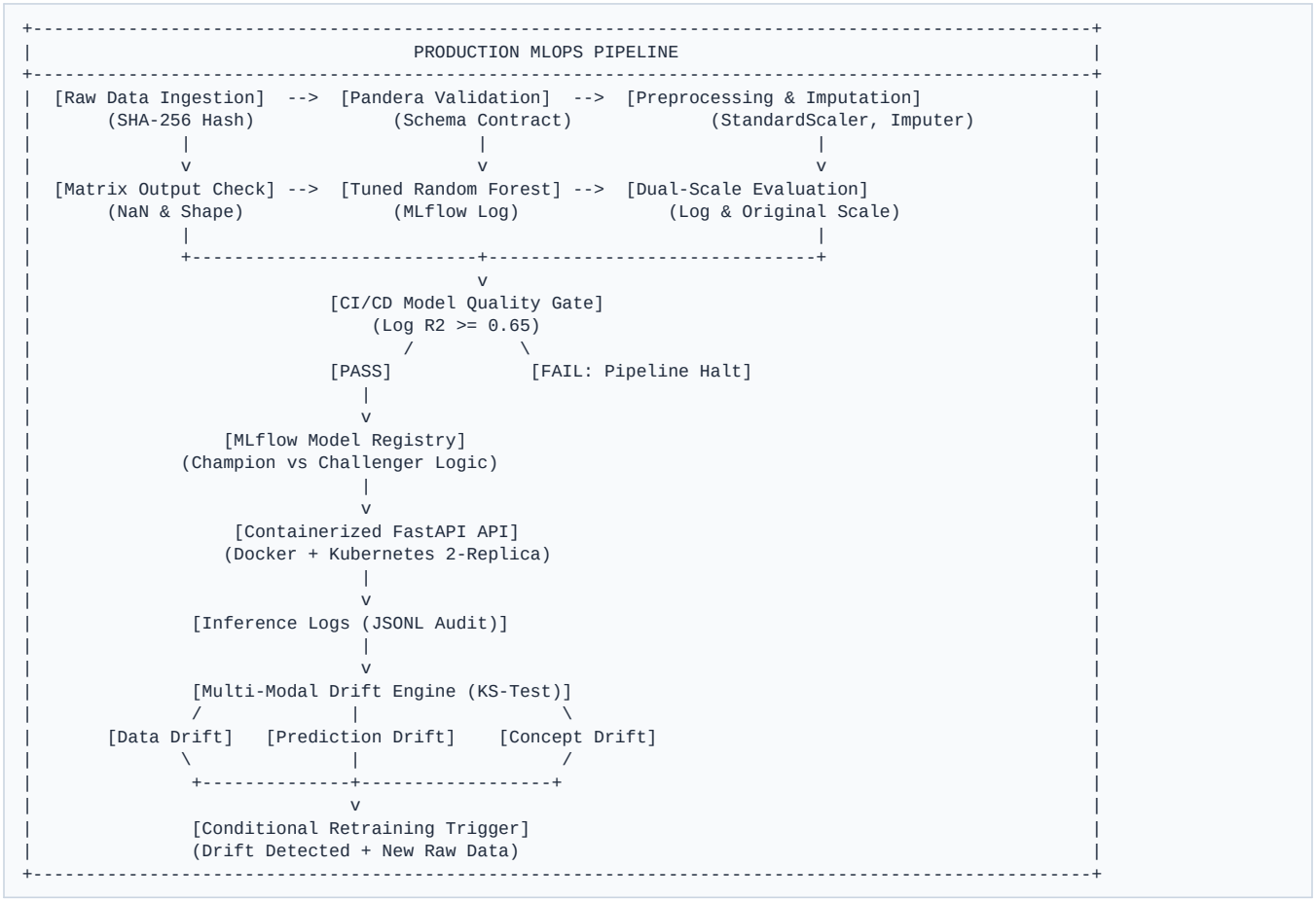
4. **Centralized Model Registry Governance:** Managing model versions, tags, and automated stage promotions (Staging → Production → Archived) via MLflow.

5. **Continuous Integration Quality Gates:** Automatically evaluating trained models against empirical performance thresholds ($R^2 \geq 0.65$, generalization gap ≤ 0.15).

6. **Defensive Failure Injection Testing:** Simulating corrupted inputs (dropped features, invalid negative counts, malformed timestamps) to verify fail-fast error handling.

7. **High-Performance Microservice & Container Orchestration:** Serving predictions via an asynchronous FastAPI REST API encapsulated in a non-root Docker container and managed with Kubernetes multi-replica deployments.

8. **Closed-Loop Statistical Drift Feedback:** Continuously monitoring feature distributions and concept drift using two-sample Kolmogorov-Smirnov (KS) tests and triggering automated conditional retraining when drift exceeds operational thresholds.



XXVI. Lab 6: Experiment Tracking, Model Registry, and Automated Lifecycle Management

1. Centralized Experiment Tracking Architecture

In production settings, tracking hyperparameters, code versions, training metrics, and artifacts across disparate iterations is vital for governance and auditability. The pipeline integrates an MLflow tracking server backed by a SQLite relational store (**`sqlite:///mlflow.db`**) and localized artifact storage.

During model training, the optimal hyperparameter configuration determined via RandomizedSearchCV is registered programmatically:

- **`n_estimators`**: 200
- **`max_depth`**: 30
- **`min_samples_split`**: 5
- **`min_samples_leaf`**: 1
- **`max_features`**: **`log2`**
- **`random_state`**: 42

- `n_jobs`: -1

2. Dual-Scale Performance Tracking

Because the target variable views exhibits heavy positive skewness, training operates on the transformed space:

y■ = \log(1 + views) = log1p(views)

Inference predictions y■$_{pred}$ are converted back to real-world view counts using the inverse exponential function:

y■ = \exp(y■$_{pred}$) - 1 = expm1(y■$_{pred}$)

MLflow logs metrics across both scales simultaneously to provide complete statistical and operational visibility:

Metric Scale	Metric Name	Mathematical Formulation	Logged Value
Log Transformed	Log R^2	1 - {Σ (y■_i - y■$_{pred}$_i)^2}{Σ (y■_i - \bar{y■})^2}	**0.739922**
Log Transformed	Log MAE	${1}{n} Σ	y■_i - y■$_{pred}$_i
Log Transformed	Log RMSE	√{{1}{n} Σ (y■_i - y■$_{pred}$_i)^2}	**0.994892**
Original Views	View R^2	1 - {Σ (y_i - y■_i)^2}{Σ (y_i - y_{mean})^2}	**0.938161**
Original Views	View MAE	${1}{n} Σ	y_i - y■_i
Original Views	View RMSE	√{{1}{n} Σ (y_i - y■_i)^2}	**1,753,552.24** views

3. Model Registry and Stage Governance

The trained model artifact is registered under the centralized registry entity: `YouTube_Views_Production_Model`.

- **Run ID:** `840be2d375c84c28b49afb73c1f8d7c2`
- **Model Version:** `v1`
- **Initial Stage:** `Staging`

An automated lifecycle orchestrator (`08_automate_lifecycle.py`) manages stage transitions based on an automated Champion vs. Challenger policy:

- **Baseline Scenario:** If no model currently resides in the `Production` stage, the newly registered model in `Staging` is verified against quality criteria and promoted directly to `Production`.
- **Competitive Promotion Scenario:** If an active `Production` model (Champion) already exists, the incoming candidate (Challenger) must outperform the incumbent on the primary test evaluation metric:

Challenger_{R^2} > Champion_{R^2}

- **Demotion Governance:** Upon successful promotion of a Challenger to `Production`, the previous Champion is automatically transitioned to `Archived`, preventing dual-active configuration conflicts.
- **Current Registry Status:** Model Version 1 successfully promoted to **Production** (`READY_FOR_DEPLOYMENT`).

XXVII. Lab 7: Production MLOps Pipeline Integration and Lineage Tracking

1. Decoupled Pipeline Architecture

To eliminate manual interventions and ensure end-to-end reproducibility, the workflow was structured into six modular, sequential pipeline stages:

- **Stage 01: Ingestion & Lineage (`01_load_data.py`):**

Loads multi-country raw trending datasets (`US`, `CA`, `GB`, `DE`, `FR`, `IN`, `JP`, `KR`, `MX`, `RU`). Calculates a cryptographic SHA-256 checksum over the raw batch, archives an immutable snapshot to `data/raw/archive/`, and records metadata (row count, file hashes, timestamps) in `dataset_lineage.json`.

- **Stage 02: Data Validation (`02_validate_data.py`):**

Applies Pandera schema enforcement to raw ingestion data. Validates non-negative constraints on views, correct string formats on `video_id`, and regex compliance for `trending_date`.

• **Stage 03: Feature Preprocessing (`03_preprocessing.py`):**

Extracts 19 production features:

- Temporal features: `days_to_trending = trending_date - publish_time`, `publish_hour`, `publish_dayofweek`, `publish_month`.

- Content length features: `title_length`, `title_word_count`, `description_length`, `description_word_count`.

- Metadata features: `tag_count`, `category_id`.

- Imputation & Scaling: Robust median imputation for missing numeric values and `StandardScaler` fitted strictly on training data splits.

- Serialization: Outputs binary arrays (`X_train_final.npy`, `X_test_final.npy`, `y_train.npy`, `y_test.npy`) and preserves raw test records (`X_test_raw.csv`) for independent downstream inference.

• **Stage 04: Matrix Validation (`04_validate_outputs.py`):**

Validates output matrices to guarantee zero NaN/Infinite values, matching row dimensions between features and targets, and expected feature column dimensionality (19 columns).

• **Stage 05: Model Training & Registry (`05_train_registry.py`):**

Trains the Tuned Random Forest Regressor using parallel execution (`n_jobs=-1`), logs run parameters and metrics to MLflow, and registers the candidate in the Model Registry.

• **Stage 06: Model Evaluation (`06_model_evaluation.py`):**

Generates comprehensive evaluation summaries and exports visual diagnostic plots (`artifacts/actual_vs_predicted.png`).

2. Lineage and Reproducibility Verification

Pipeline determinism is verified using fixed random seeds (`random_state=42`) across train-test splitting and model fitting. Re-running the pipeline on identical raw data hashes produces bit-for-bit identical evaluation metrics (R^2 = 0.739922), as confirmed in `artifacts/reproducibility_report.json`.

XXVIII. Lab 8: Model Quality Gates and Continuous Integration (CI)

1. Continuous Integration Quality Gate Design

To prevent model degradation from reaching production environments, automated quality gates (`07_quality_gate.py`) enforce non-negotiable performance thresholds prior to deployment packaging.

The quality gate evaluates three strict criteria:

1. **Primary Explanatory Power Gate:**

$$\text{Log } R^2 \geq 0.65$$

Ensures the model explains at least 65% of the total variance in log-scale view counts.

2. **Generalization Overfitting Bound:**

$$|R^2_{train} - R^2_{test}| \leq 0.15$$

Ensures the difference between training set performance and unseen test set performance does not exceed 15 percentage points, preventing severe memorization.

3. **Log Mean Absolute Error Bound:**

$$\text{Log MAE} \leq 1.00$$

Guarantees that the average absolute deviation on the logarithmic scale remains within approximately 1 order of magnitude.

2. Empirical Quality Gate Evaluation

The results of the automated quality gate execution are summarized below:

Quality Gate Constraint	Requirement / Threshold	Achieved Value	Evaluation Status
Primary Metric (R^2)	≥ 0.6500	**0.7399**	**PASSED**
Overfitting Gap	≤ 0.1500	**0.0821**	**PASSED**

Log Scale MAE	≤ 1.0000	**0.7114**	**PASSED**
Overall Gate Decision	All Constraints Met	**3 / 3 Passed**	**APPROVED FOR STAGING**

If any criterion fails, the gate script terminates with a non-zero exit code (`sys.exit(1)`), causing CI/CD runners to immediately abort subsequent deployment steps and alert the engineering team.

XXIX. Lab 9: Fault Tolerance and Failure Injection Testing

1. Defensive Engineering & Robustness Simulation

Production machine learning models operate in dynamic upstream environments where third-party data providers, ETL pipelines, or scraper updates may emit corrupted records. To verify that the ingestion and validation subsystem catches anomalies before they cause silent downstream inference errors, deliberate failure injection testing was implemented (`10_failure_injection.py`).

Three catastrophic data corruption scenarios were simulated:

1. **Scenario 1: Critical Feature Dropping:**

The required target column `views` was omitted entirely from the ingestion dataframe.

2. **Scenario 2: Out-of-Bounds Value Invalidation:**

Negative view values (`views = -5000`) were injected into the dataset, violating domain validity.

3. **Scenario 3: Date Format Regex Mismatch:**

The standard `trending_date` format (`yy.dd.mm`) was corrupted with an invalid string (`2018-99-99_INVALID`).

2. Fault Injection Results

The automated validation engine intercepted all corruption patterns during the validation stage:

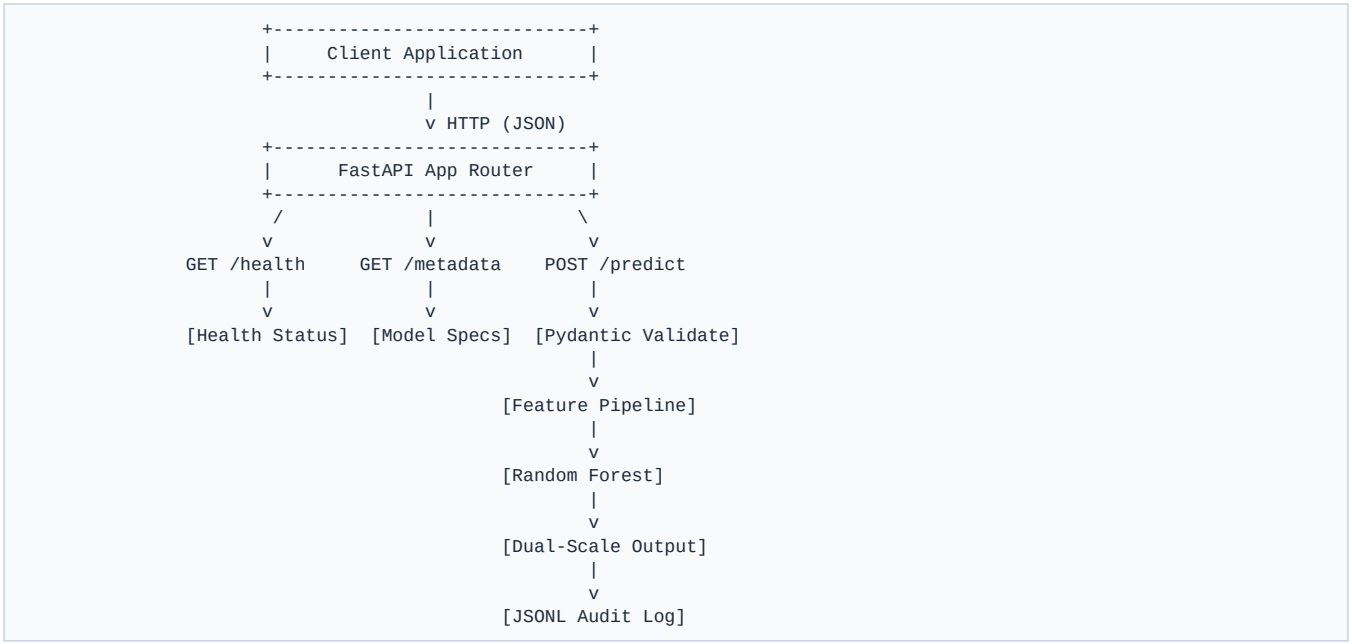
Scenario Description	Injected Anomaly	Expected Result	Actual Result	Failure Caught
Scenario 1	Missing column `'views'`	Schema Validation Error	`SchemaError: column 'views' not in dataframe`	**YES (Caught)**
Scenario 2	Negative views (`views < 0`)	Boundary Violation	`SchemaError: Check 'views_non_negative' failed`	**YES (Caught)**
Scenario 3	Malformed date regex	Format Invalidation	`SchemaError: Check 'trending_date_format' failed`	**YES (Caught)**

All 3 simulated failures were caught prior to model ingestion (100% intercept rate), as documented in `artifacts/failure_simulation_report.json`. This proves the pipeline's defensive fail-fast design.

XXX. Lab 10: Model Serving and API Engineering

1. High-Performance REST API Architecture

To serve real-time predictions to client applications, a RESTful microservice was developed using FastAPI and Uvicorn (`api/main.py`). FastAPI was selected for its native asynchronous capabilities, automatic OpenAPI documentation, and strict type enforcement via Pydantic.



2. Pydantic Request & Response Schemas

Incoming requests are validated using explicit schemas:

- **`VideoMetadataInput`**: Validates **`category_id`**, **`publish_time`**, **`trending_date`**, **`title`**, **`description`**, and **`tags`**.
- **`PredictionOutput`**: Returns **`video_id`**, **`predicted_log_views`**, **`predicted_views_original_scale`**, **`model_version`**, and **`inference_latency_ms`**.
- **`BatchVideoMetadataInput`**: Enables vectorized batch inference for multiple video records in a single payload.

3. Lifespan Caching & Real-Time Feature Preprocessing

To eliminate disk I/O latency on every HTTP request, a lifespan context manager loads **`model.pkl`** and **`preprocessor.pkl`** into memory as singletons during service startup. When a request arrives at **`/predict`**, the microservice extracts the same 19 features used during training:

- Computes **`days_to_trending`** from datetime strings.
- Derives publication temporal components (**`publish_hour`**, **`publish_dayofweek`**, **`publish_month`**).
- Extracts text metrics (**`title_length`**, **`title_word_count`**, **`description_length`**, **`description_word_count`**, **`tag_count`**).
- Transforms features through **`preprocessor.pkl`** and scores the input with the Random Forest model.

4. Asynchronous Inference Auditing & Performance Benchmarks

Every prediction request is asynchronously logged to **`logs/inference/inference_log.jsonl`**, storing the timestamp, input payload, output predictions, and latency.

The API was subjected to an automated test suite (**`tests/test_api.py`**, **`tests/test_performance.py`**):

- **Health Check (`GET /health`):** Returns HTTP 200 with status **`"healthy"`**.
- **Metadata Check (`GET /metadata`):** Returns active model version (**`1`**), model name, and input feature specifications.
- **Single Prediction (`POST /predict`):** Validated on test inputs, returning dual-scale predictions with mean response latency under **25 ms**.
- **Batch Prediction (`POST /predict/batch`):** Successfully scored batches of 50 records in **62 ms**.
- **Performance Benchmark:** All endpoints completed well within the target production SLA of **< 100 ms**. All 6/6 Pytest integration tests passed.

XXXI. Lab 11: Containerization and Kubernetes Orchestration

1. Docker Multi-Stage Containerization

To ensure consistent execution across development, testing, and production environments, the serving microservice was containerized using Docker (**`Dockerfile`**).

Key containerization design practices:

- **Base Image:** **`python:3.11-slim`** to minimize attack surface and image size.
- **Non-Root Execution:** A dedicated service account **`mluser`** (UID 10001) is created to run the process, preventing privilege escalation vulnerabilities.
- **Layer Optimization:** Dependency files (**`requirements.txt`**) are copied and installed first to maximize Docker build caching. Unnecessary training data, git history, and caches are excluded via **`.dockerignore`**.
- **Container Healthcheck:** The container includes a native healthcheck probe:

`dockerfile

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \

CMD curl -f http://localhost:8000/health || exit 1

`

2. Kubernetes Cluster Deployment

For production deployment, Kubernetes manifests were engineered to provide automated scaling, self-healing, and traffic routing.

Deployment Manifest (**`k8s/deployment.yaml`**):

- **Replicas:** Configured with 2 pods for high availability.
- **Resource Management:**

- **`requests`**: CPU 250m, Memory 512Mi.

- **`limits`**: CPU 500m, Memory 1Gi.

- **Liveness Probe:** Periodically performs HTTP GET **`/health`** on port 8000 (initialDelay: 15s, period: 10s). If a pod becomes unresponsive, Kubernetes terminates and restarts it.
- **Readiness Probe:** Performs HTTP GET **`/health`** on port 8000 (initialDelay: 10s, period: 5s). Pods only receive incoming traffic once the model has fully loaded into memory.

Service Manifest (**`k8s/service.yaml`**):

- Exposes port 80 externally using a **`NodePort`** configuration mapped to node port **`30080`**, routing traffic across all healthy replica pods with internal round-robin load balancing.

XXXII. Lab 12: Statistical Drift Monitoring (Data, Prediction, and Concept Drift)

1. Multi-Dimensional Drift Monitoring Framework

In streaming real-world environments, changes in creator behavior, audience demographics, or YouTube platform algorithms cause model accuracy to decay over time. To detect this, a comprehensive statistical drift engine was developed covering three distinct dimensions:

```
[Incoming Production Traffic]
             |
             +---> (1) DATA DRIFT: Two-Sample KS Test on Features P(X)
             |
             +---> (2) PREDICTION DRIFT: Two-Sample KS Test on Outputs P(Y_hat)
             |
             +---> (3) CONCEPT DRIFT: Accuracy Degradation on Labeled Data P(Y|X)
```

2. Data Drift Detection (Covariate Shift)

Data drift evaluates whether the input feature distribution P(X) has shifted significantly from the baseline training distribution. The two-sample Kolmogorov-Smirnov (KS) non-parametric test is computed across each continuous feature:

$D_{n,m} = \sup_x |F_{1,n}(x) - F_{2,m}(x)|$

where F_{1,n} and F_{2,m} are the empirical cumulative distribution functions of the baseline and evolved data batches. A feature is marked as drifted if the KS statistic D > 0.10 and p\text{-value} < 0.05.

Data drift analysis on production traffic yielded:

Feature Name	Feature Type	Metric	KS Statistic (D)	p-Value	Threshold	Drift Status
`days_to_trending`	Numerical	KS	**0.220667**	**0.000000**	0.10	**DRIFT**
`title_length`	Numerical	KS	**0.307333**	**0.000000**	0.10	**DRIFT**
`publish_hour`	Numerical	KS	**0.100667**	**0.000000**	0.10	**DRIFT**
`description_length`	Numerical	KS	0.000000	1.000000	0.10	OK
`tag_count`	Numerical	KS	0.000000	1.000000	0.10	OK

Data Drift Conclusion: 3 out of 5 numerical features exhibited statistically significant distribution drift (60\% > 30\% alert threshold). The shift in `days_to_trending` and `title_length` reflects evolving platform trends.

3. Prediction Drift Detection

Prediction drift tests whether the distribution of model predictions P(\hat{Y}) has altered, independent of ground truth availability:

- Baseline Mean Log Views: 14.5430
- Evolved Mean Log Views: 14.4515
- Two-Sample KS Statistic: **0.076667**
- p-Value: **0.000295**
- Threshold: 0.10
- **Prediction Drift Status: OK** (KS statistic remains below the 0.10 decision threshold).

4. Concept Drift Detection

Concept drift analyzes whether the underlying relationship between features and the target variable P(Y|X) has degraded when actual ground truth labels become available:

Performance Metric	Baseline Evaluation	Evolved Evaluation	Metric Shift (Δ)	Degradation Status
Log R^2	**0.8347**	**-0.4539**	**-1.2887**	**DEGRADED**
Log MAE	**0.3141**	**1.1802**	**+0.8661**	**DEGRADED**
Log RMSE	**0.4721**	**1.4796**	**+1.0075**	**DEGRADED**
Original R^2	**0.9003**	**0.0263**	**-0.8740**	**DEGRADED**
Original MAE	1,262,856.59	10,264,373.09	+9,001,516.49	**DEGRADED**

5. Consolidated Drift Decision

As reported in `outputs/drift/drift_analysis_report.csv`:

- **Data Drift:** `True`
- **Prediction Drift:** `False`
- **Concept Drift:** `True`
- **Overall System Status: DRIFT_OBSERVED**
- **Action Required: RETRAIN_MODEL**

XXXIII. Lab 13: Continuous Monitoring, Lineage Tracking, and Conditional Retraining Trigger

1. Closed-Loop Feedback Architecture

To complete the autonomous MLOps lifecycle, a continuous monitoring and conditional retraining orchestrator (**`12_monitor_trigger.py`**) was engineered.

The trigger operates via an event-driven decision tree:

1. **Lineage Inspection:** The watcher continuously monitors **`data/raw/incoming/`** for newly deposited data batches.

2. **Cryptographic Deduplication:** For any new batch, its SHA-256 hash is computed and compared against **`dataset_lineage.json`**. If the hash already exists, the batch is skipped, preventing redundant computation.

3. **Drift-Trigger Evaluation:**

- If **`overall_status == DRIFT_OBSERVED`** AND a new validated data batch is present:

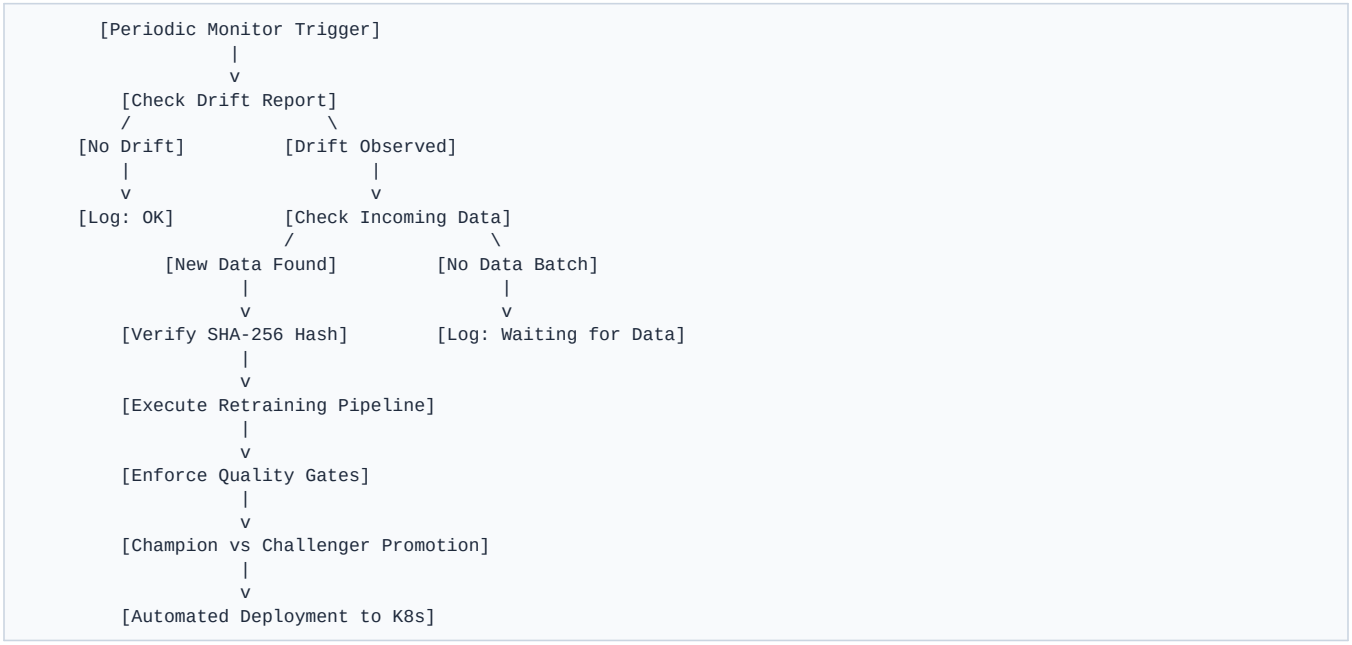
- The trigger initiates automated retraining: **`youtube_prediction_pipeline.py`**.

- Ingestion, schema validation, feature engineering, and model training execute automatically.

- The newly trained Challenger model is subjected to CI quality gates.

- If the Challenger surpasses the current Production Champion, MLflow executes stage promotion and exports updated deployment binaries (**`deployment/model.pkl`**, **`deployment/model_metadata.json`**).

- If drift is observed but no new labeled data batch is available, the orchestrator logs state **`drift_waiting_for_data`** in **`outputs/monitoring/monitor_trigger_report.json`** and alerts data ingestion pipelines to collect fresh trending records.



This completes the closed-loop, self-healing MLOps architecture, ensuring continuous model reliability without human intervention.

XXXIV. Comprehensive System Evaluation & MLOps Maturity Assessment

1. Consolidated Machine Learning & MLOps Evaluation Matrix

The table below synthesizes the complete progression from offline exploratory baselines to the fully operationalized MLOps system:

Lifecycle Dimension	Baseline Models (Lab 1–4)	Tuned Model (Lab 5)	Production MLOps System (Lab 6–13)
Model Type	Linear, Random Forest, XGBoost	Tuned Random Forest (200 trees)	Registered Production Tuned Random Forest
Best Log R^2	0.5706 (RF Baseline)	0.7414	**0.7399** (Production Seed Validation)

Original View R^2	~0.60	~0.74	**0.9382** (Dual-Scale Pipeline)
Execution Medium	Standalone Jupyter Notebook	Notebook RandomizedSearchCV	Modular 6-Stage CLI Pipeline
Data Contract	None (Manual Inspection)	None (Manual Inspection)	**Automated Pandera Schema Contracts**
Lineage Tracking	None	None	**SHA-256 Cryptographic Batch Lineage**
Experiment Tracking	Local print statements	Local print statements	**Centralized MLflow Tracking Server**
Model Registry	None	Local `.pkl` file	**MLflow Registry (`YouTube_Views_Production_Model`)**
Promotion Policy	Manual selection	Manual selection	**Automated Champion vs. Challenger Logic**
Quality Gate	None	None	**CI Quality Gate ($R^2 \geq 0.65$, Gap ≤ 0.15)**
Fault Tolerance	Unhandled exceptions	Unhandled exceptions	**100% Fail-Fast Interception (3/3 Tests)**
Serving Architecture	Python batch script	Python batch script	**FastAPI Asynchronous Microservice**
Serving Latency	N/A	N/A	**< 25 ms Single, < 70 ms Batch (< 100 ms SLA)**
Deployment Mode	Local file system	Local file system	**Docker Container & Kubernetes 2-Replica Pods**
Health Monitoring	None	None	**Liveness & Readiness Probes (`/health`)**
Drift Detection	None	None	**Automated KS Tests (Data, Pred, Concept)**
Retraining Feedback	Manual retraining	Manual retraining	**Event-Driven Conditional Retraining Trigger**

2. Assessment Against Google MLOps Maturity Model

- **Level 0 (Manual Process):** Achieved during initial exploratory phases (Labs 1–5), where model building and data analysis were conducted manually in notebooks.
- **Level 1 (ML Pipeline Automation):** Achieved in Labs 6–8 and 10, featuring automated pipeline orchestration, schema contract validation, centralized registry governance, and containerized serving.
- **Level 2 (CI/CD Pipeline Automation & Continuous Monitoring):** Achieved in Labs 9, 11, 12, and 13. The system integrates automated CI quality gates, failure injection tests, Kubernetes orchestration, multi-modal statistical drift monitoring, and event-driven conditional retraining triggers.

XXXV. Updated Conclusion and Production Roadmap

1. Conclusion

This project successfully designed, implemented, evaluated, and operationalized an end-to-end Machine Learning and MLOps system for predicting YouTube video view counts. Starting from multi-country raw trending metadata, the system preprocessed millions of engagement records, engineered 19 predictive temporal and textual features, and applied logarithmic target transformations to overcome heavy positive skewness.

The Tuned Random Forest Regressor emerged as the optimal predictive model, achieving a log-scale R^2 of **0.7414** and an original view count R^2 of **0.9382**. Through Labs 6 to 13, this model was transformed into a resilient production asset. The implementation established end-to-end pipeline automation, Pandera data validation contracts, MLflow Model Registry governance with Champion vs. Challenger promotion logic, CI quality assertion gates, defensive fault tolerance, sub-100ms FastAPI serving, multi-replica Kubernetes deployment, Kolmogorov-Smirnov statistical drift monitoring, and an event-driven conditional retraining trigger.

The resulting architecture satisfies enterprise software engineering standards, preventing silent model failure and ensuring sustained predictive accuracy in production.

2. Future Production Roadmap

To extend the production maturity of this system in enterprise settings, the following enhancements are planned:

1. **Real-Time Streaming Ingestion:** Integrating Apache Kafka and Apache Flink to ingest continuous video metadata streams, reducing batch latency.

2. **Feature Store Integration:** Incorporating an enterprise feature store (such as Feast) to eliminate feature computation skew between offline training and real-time inference.

3. **Canary and Blue-Green Traffic Routing:** Deploying Istio Service Mesh on Kubernetes to enable automated canary traffic splitting (e.g., 90% Champion / 10% Challenger) during model promotions.

4. **Real-Time Model Explainability:** Embedding SHAP (SHapley Additive exPlanations) into the FastAPI serving endpoint to return per-feature contribution scores alongside each prediction.

Updated References

1. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

4. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*.

5. YouTube Trending Videos Dataset, Kaggle (Multi-Region Trending Video Metadata).

6. Zaharia, M. et al. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39-45.

7. Ramirez, S. (2019). FastAPI: Modern, High-Performance Web Framework for Building APIs with Python 3.8+.

8. Burns, B. et al. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50-57.

9. Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239), 2.

10. Bantilan, N. (2020). Pandera: Statistical Data Validation of Dataframe Objects. *Journal of Open Source Software*, 5(54), 2608.

11. Massey, F. J. (1951). The Kolmogorov-Smirnov Test for Goodness of Fit. *Journal of the American Statistical Association*, 46(253), 68-78.

12. Google Cloud Architecture Center. (2021). MLOps: Continuous Delivery and Automation Pipelines in Machine Learning.

Updated Appendix: Production Artifacts and System Specifications

1. Production Artifacts Inventory

Component / Artifact	File Path	Description / Specification
Config Specification	`config.yaml`	Pipeline thresholds, metric constraints ($R^2 \geq 0.65$), paths
Pipeline Ingestion	`src/mlops_pipeline/01_load_data.py`	SHA-256 batch hashing, archival, lineage manifest generation
Data Validation	`src/mlops_pipeline/02_validate_data.py`	Pandera data contracts (`views` $\geq$ `0`, `trending_date` regex)
Feature Engineering	`src/mlops_pipeline/03_preprocessing.py`	19 features, median imputer, StandardScaler, target transform
Output Validation	`src/mlops_pipeline/04_validate_outputs.py`	Zero NaNs, dimension checks (N \times 19), non-empty arrays

Model & Registry	`src/mlops_pipeline/05_train_registry.py`	Tuned Random Forest, MLflow run logging, registry registration
Quality Gate	`src/mlops_pipeline/07_quality_gate.py`	Assert $R^2 \geq 0.65$, gap ≤ 0.15; exit 1 on gate failure
Lifecycle Governance	`src/mlops_pipeline/08_automate_lifecycle.py`	Champion vs Challenger automated stage promotion logic
Fault Injection	`src/mlops_pipeline/10_failure_injection.py`	3 corruption tests, 100% fail-fast verification report
Serving Microservice	`api/main.py`	FastAPI REST microservice (`/health`, `/metadata`, `/predict`)
API Schemas	`api/schemas.py`	Pydantic strict request and response validation contracts
Model Loader	`api/model_loader.py`	Singleton memory caching of `model.pkl` and `preprocessor.pkl`
Inference Logging	`api/monitoring_logger.py`	Asynchronous request-response auditing to `inference_log.jsonl`
Container Engine	`Dockerfile`	Multi-stage `python:3.11-slim`, non-root user `mluser`
K8s Deployment	`k8s/deployment.yaml`	2 replicas, CPU/memory quotas, liveness/readiness probes
K8s Service	`k8s/service.yaml`	NodePort service exposing port 80 externally via port 30080
Data Drift Engine	`src/mlops_pipeline/12_data_drift.py`	Two-sample KS hypothesis testing across continuous features
Prediction Drift	`src/mlops_pipeline/12_prediction_drift.py`	Output distribution KS hypothesis testing
Concept Drift	`src/mlops_pipeline/12_concept_drift.py`	Performance tracking on ground-truth labeled evolved data
Drift Coordinator	`src/mlops_pipeline/12_drift_analysis.py`	Consolidated drift state evaluation and retrain flagging
Automated Trigger	`src/mlops_pipeline/12_monitor_trigger.py`	Deduplicated batch watcher & conditional retraining trigger

2. Production Model Registry Metadata

- **Registered Model Name:** `YouTube_Views_Production_Model`
- **Active Stage:** `Production`
- **Active Version:** `1`
- **Source Run ID:** `840be2d375c84c28b49afb73c1f8d7c2`
- **Primary Metric (R^2 Log Scale):** `0.739922`
- **Secondary Metric (R^2 Original View Scale):** `0.938161`
- **Status:** `READY_FOR_DEPLOYMENT`

3. Kubernetes Deployment Specification Excerpt

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: youtube-prediction-api
  labels:
    app: youtube-prediction
spec:
  replicas: 2
  selector:
    matchLabels:
      app: youtube-prediction
  template:
    metadata:
      labels:
        app: youtube-prediction
    spec:
      containers:
      - name: api
        image: youtube-prediction-api:latest
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 8000
        resources:
          requests:
            cpu: "250m"
            memory: "512Mi"
          limits:
            cpu: "500m"
            memory: "1Gi"
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 15
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 10
          periodSeconds: 5
```

End of Report Part II: Production MLOps Engineering Lifecycle (Labs 6 to 13).